

Smart Grid ML

This can be turned into a **very strong FYP**, but the important thing is to define it correctly.

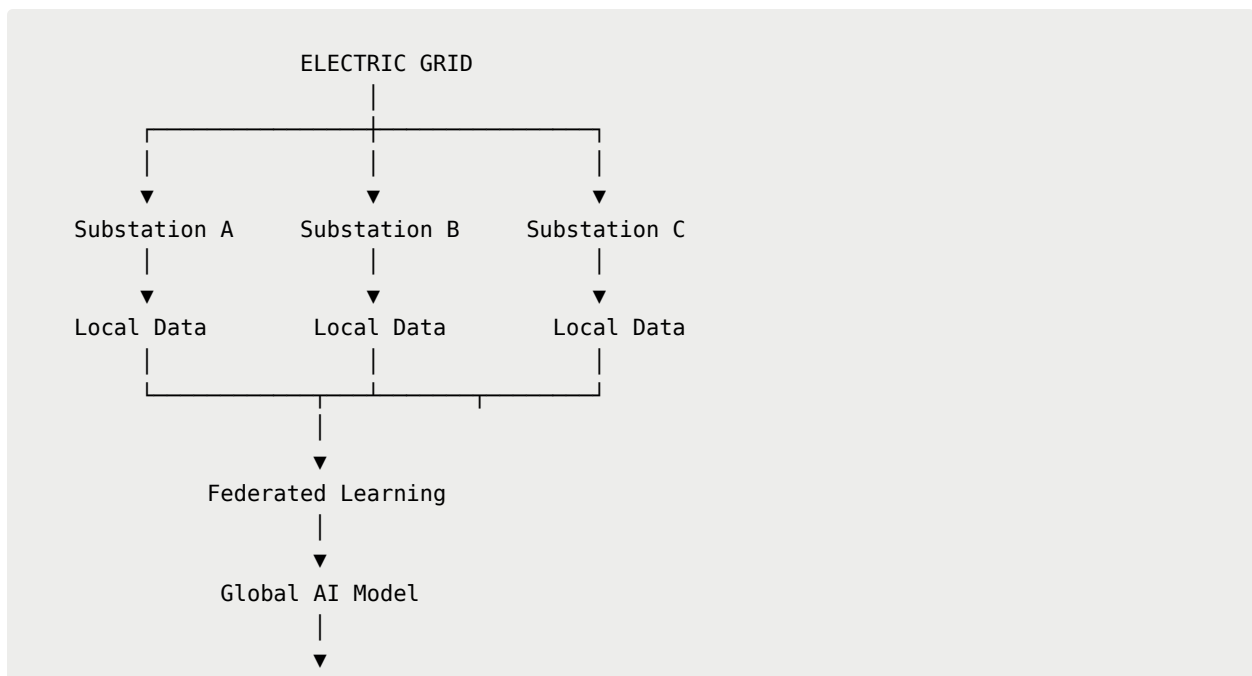
You are **not building a real electricity grid**. You are building a **software-based intelligent grid laboratory** in which several simulated substations behave as independent organizations/nodes, each having its own local electricity/asset data. AI models learn locally, a federated-learning server aggregates model updates, and a group of specialized AI agents analyze the resulting information and recommend actions to a human operator.

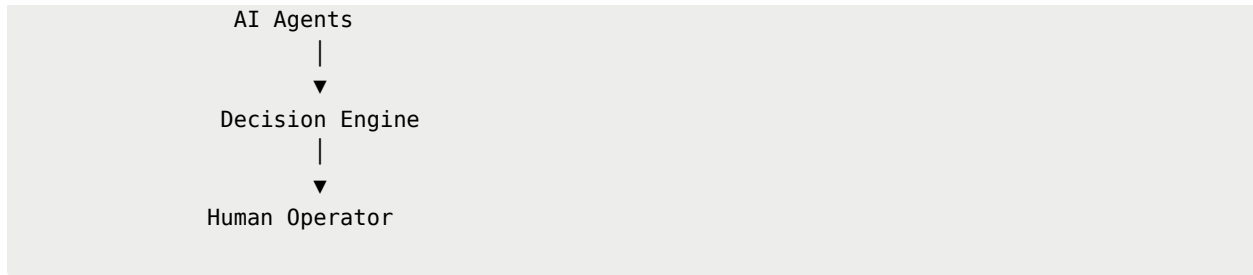
The impressive part is that this combines:

AI/ML + Federated Learning + Agentic AI + Explainable AI + Cybersecurity + IoT concepts + Time-Series Forecasting + Predictive Maintenance + Smart Grid

1. First understand the project in very simple words

Imagine an electricity company has four substations:





Each substation has its **own data**.

For example:

Substation A

Time	Load	Voltage	Current	Temp
10:00	82 kW	231 V	356 A	55°C
10:01	84 kW	230 V	365 A	56°C
10:02	91 kW	229 V	397 A	58°C
...				

Substation B has different data.

Substation C has different data.

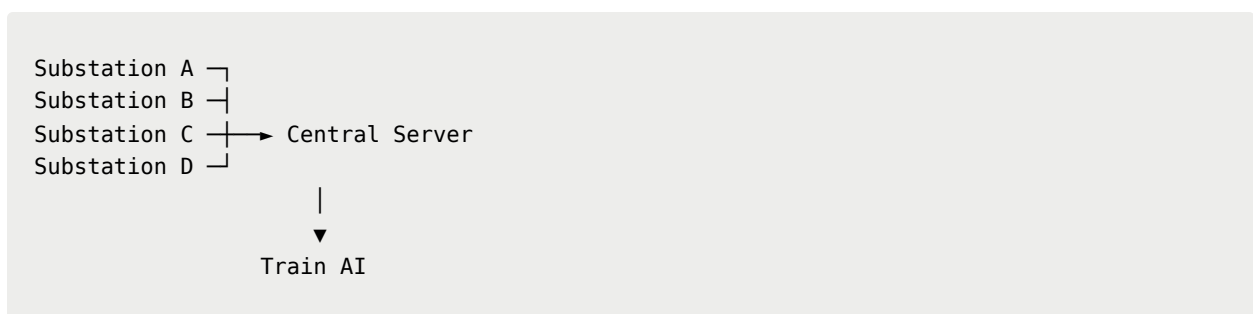
Instead of sending all raw data to one central server, each substation trains an ML model **locally**.

Only the learned model updates are sent to the central federated server.

That's the key idea.

2. Why Federated Learning?

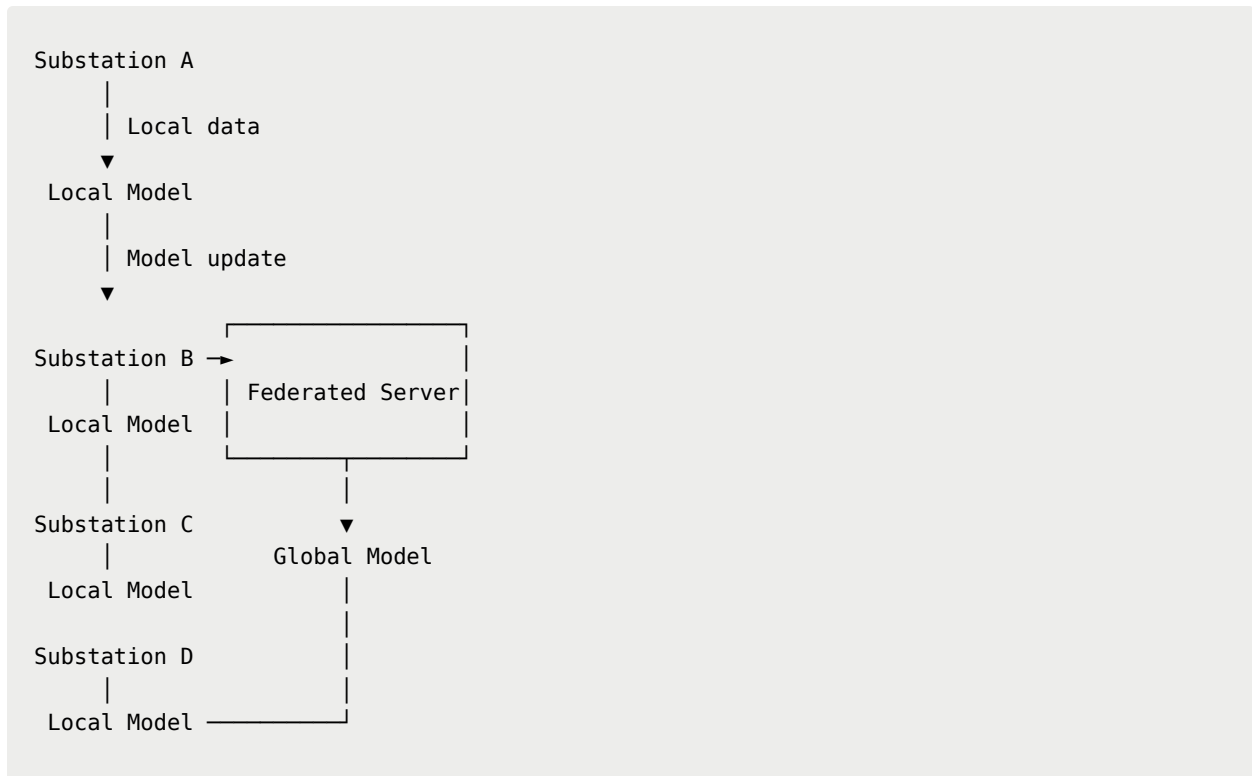
Normally, machine learning works like this:



The problem is that the central server receives everyone's raw data.

In a real utility environment, that can be undesirable because operational data may be sensitive.

Federated Learning changes the architecture:



The raw data **stays at the substation**.

The server receives things such as:

Model weights
Gradients
Training statistics

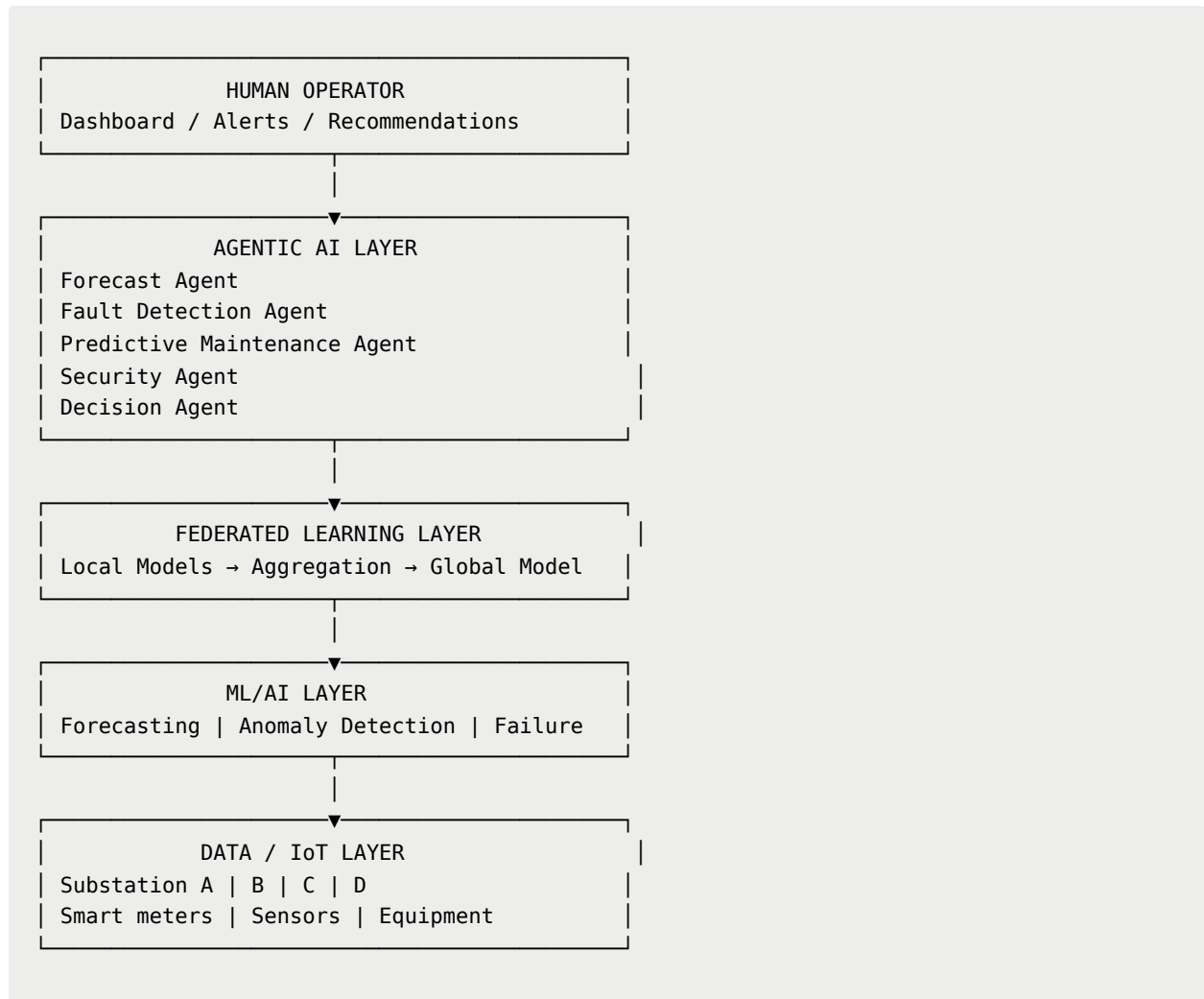
and combines them.

3. What exactly is your FYP building?

I would define your system as:

A privacy-preserving multi-agent AI platform for distributed smart-grid monitoring, forecasting, anomaly detection and predictive maintenance using federated learning.

Your system will have approximately **five major layers**.



4. What is a "substation" in your project?

Don't worry about the electrical engineering complexity initially.

For your FYP, think of a substation as a **data-producing node**.

For example:

SUBSTATION A

Transformer T-01
Transformer T-02
Smart Meter 1
Smart Meter 2
Temperature Sensor
Voltage Sensor
Current Sensor
Vibration Sensor

It produces data.

Your software simulates this.

You don't actually need a physical transformer.

5. What data does a substation generate?

You can divide the data into several categories.

A. Electrical measurements

Voltage
Current
Active Power
Reactive Power
Frequency
Power Factor
Energy Consumption

Example:

Voltage = 231.4 V
Current = 410 A
Active Power = 91.5 kW
Frequency = 49.98 Hz
Power Factor = 0.94

B. Equipment measurements

For transformers:

```
Transformer temperature
Oil temperature
Vibration
Load percentage
Operating hours
```

Example:

```
Temperature = 82°C
Vibration = 4.2 mm/s
Load = 91%
```

C. Renewable generation

If your substation contains solar:

```
Solar generation
Solar voltage
Solar current
Solar irradiance
```

Example:

```
Solar Generation = 43 kW
Irradiance = 780 W/m2
```

D. Historical information

```
Previous failures
Previous anomalies
Maintenance records
```

Equipment age
Operating history

This becomes useful for predictive maintenance.

6. Where will you get the dataset?

This is one of the most important questions your advisor will ask.

You **do not need to create everything from scratch**.

You can combine public datasets with simulated smart-grid/asset data.

Dataset 1 — UCI electricity dataset

The UCI Individual Household Electric Power Consumption dataset contains more than **2 million measurements**, sampled at one-minute intervals over almost four years, with variables including active/reactive power, voltage, current and sub-metering.

UCI Individual Household Electric Power Consumption Dataset

This is excellent for your:

- demand forecasting
 - load prediction
 - anomaly detection
 - electricity-consumption experiments
-

7. Another very useful dataset: Pecan Street

Pecan Street's Dataport contains high-resolution residential electricity data, including circuit-level electricity consumption and generation. Their datasets can include solar, EV charging, HVAC, water heaters and other loads.

Importantly, Pecan Street currently provides some free public datasets, while broader academic access is available to qualifying university researchers for non-commercial educational/research purposes.

Pecan Street Dataport

This could give you more realistic data for:

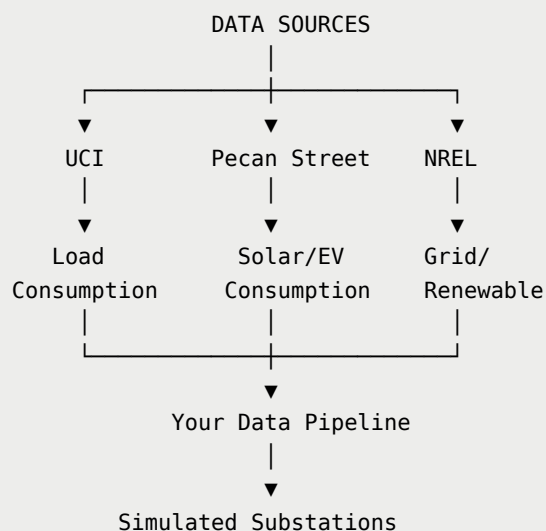
Solar
EV
Household load
HVAC
Appliances
Generation

8. NREL can provide additional energy/grid resources

NREL provides publicly available datasets and tools for renewable-energy and grid analysis, including grid modernization, solar, wind and electricity-system resources.

NREL Grid Data & Tools

So your dataset strategy can be:



9. But we need transformer failure data

This is where the project becomes interesting.

There may not be one perfect public dataset containing:

```
Transformer temperature
+
Vibration
+
Load
+
Historical failure
+
Failure timestamp
```

for exactly what your FYP needs.

So you can create a **controlled synthetic asset dataset** based on realistic relationships.

For example:

Normal transformer

```
Temperature: 55°C
Vibration: 1.2
Load: 60%
Voltage: normal
Status: NORMAL
```

Stressed transformer

```
Temperature: 78°C
Vibration: 3.7
Load: 91%
Voltage: slightly abnormal
Status: WARNING
```

Failure-risk transformer

```
Temperature: 94°C  
Vibration: 6.2  
Load: 98%  
Voltage: abnormal  
Status: CRITICAL
```

This doesn't mean you claim the data is real.

You explicitly tell your advisor:

"The electrical consumption component uses publicly available datasets, while equipment-health signals are generated using a controlled synthetic fault-injection methodology because labeled transformer failure datasets are difficult to obtain."

That is a perfectly reasonable research methodology for an FYP.

10. Your four virtual substations

This is where Federated Learning becomes easy to understand.

Suppose you create:

```
Substation A  
Substation B  
Substation C  
Substation D
```

Each receives a different portion/distribution of data.

For example:

A

Mostly residential:

```
Residential load  
Low industrial activity  
Normal weather
```

B

High commercial:

```
Commercial load  
Large daytime demand
```

C

Solar-heavy:

```
Solar generation  
Variable net load
```

D

Industrial:

```
High load  
High transformer stress
```

Now your federated-learning experiment becomes realistic because the data distributions are **different**.

This is called **non-IID data**.

11. What does non-IID mean?

This is an important concept for your FYP defense.

IID means everyone has approximately similar data.

Non-IID means:

```
A → residential  
B → commercial  
C → solar-heavy  
D → industrial
```

Therefore:

Data A $\neq$ Data B $\neq$ Data C $\neq$ Data D

This is actually desirable for your FYP because real-world organizations/substations won't necessarily have identical data distributions.

12. How Federated Learning actually works

Let's make the process extremely simple.

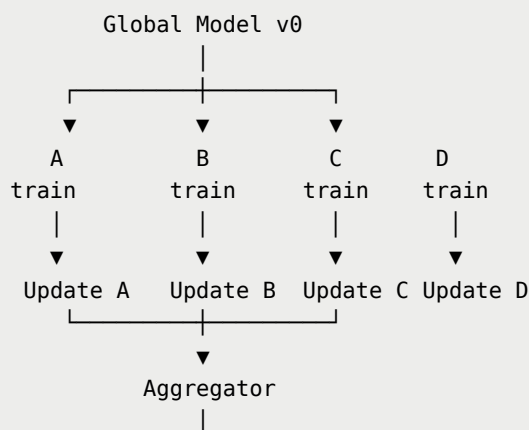
Initially:

Global Model v0

is sent to:

A
B
C
D

Each substation trains it using **only its own data**.



▼
Global Model v1

Then:

Global Model v1

is sent back.

And the process repeats.

```
Round 1
  ↓
Local training
  ↓
Aggregation
  ↓
Round 2
  ↓
Local training
  ↓
Aggregation
  ↓
Round 3
  ↓
...
```

13. What is the aggregator?

The aggregator is the central Federated Learning server.

A common method is **FedAvg — Federated Averaging**.

Conceptually:

```
Global Model =
weighted average of
local model parameters
```

For example:

Model A → 25%
Model B → 25%
Model C → 25%
Model D → 25%

↓

Global Model

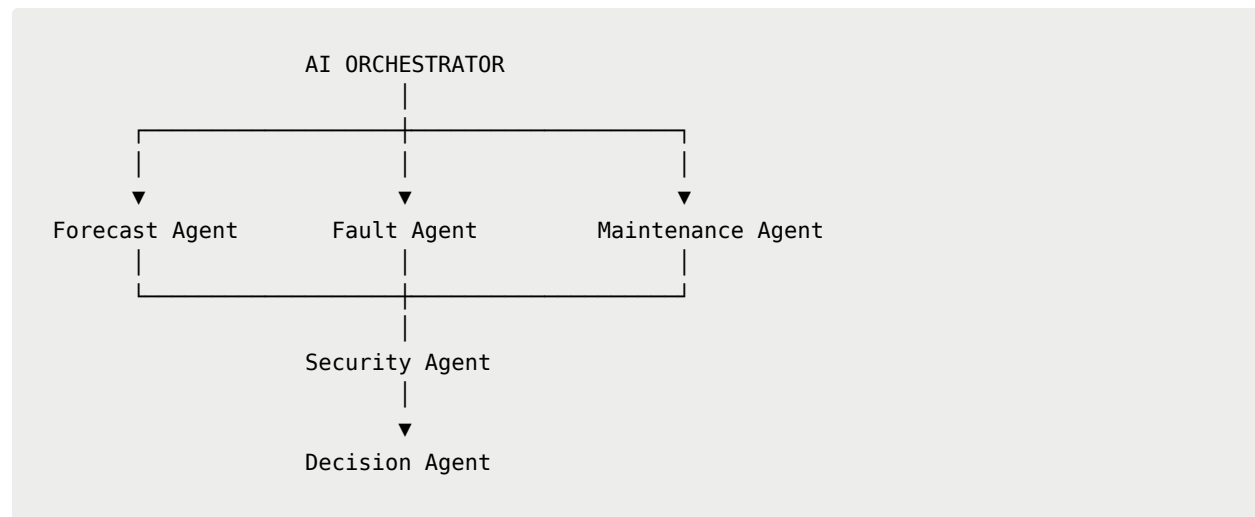
You don't need to implement the mathematics from scratch.

You can use an FL framework such as Flower, while your contribution is the **overall smart-grid architecture and intelligent agent system.**

14. Now comes the AI part

Once the global model has learned patterns, you can build several specialized AI agents.

I would recommend **five agents**, not dozens.



15. Agent 1 — Forecasting Agent

Its job:

| "What will electricity demand look like in the next few hours?"

Input:

```
Historical load
Time
Day
Temperature
Solar generation
Previous demand
```

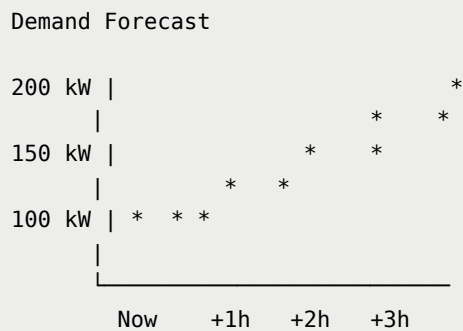
Output:

```
Next hour:
124 kW

Next 2 hours:
137 kW

Next 3 hours:
149 kW
```

Dashboard:



16. Agent 2 — Fault Detection Agent

This agent asks:

| "Is something abnormal happening?"

Suppose:

```
Normal voltage = 230–240 V
Current = normal
Temperature = normal
```

Suddenly:

```
Voltage = 198 V
Current = 490 A
Frequency = abnormal
```

The agent detects:

```
ANOMALY DETECTED
```

Example:

```
Substation C
Transformer T-17

Anomaly Score: 0.94
Severity: HIGH
```

17. Agent 3 — Predictive Maintenance Agent

This is one of the strongest demonstrations.

It asks:

| "Which equipment is likely to fail?"

Input:

```
Temperature
Vibration
Load
```

```
Age
Historical failures
Operating hours
Electrical abnormalities
```

Output:

```
Transformer T-17

Failure Probability: 87%

Risk:
HIGH
```

Then it explains:

```
Temperature anomaly    +31%
Vibration anomaly      +24%
Load increase          +19%
Historical pattern     +13%
Other                  +13%
```

This is your **Explainable AI component**.

18. Important correction about those percentages

Don't simply make the percentages up and call them "AI explanation."

Instead, use a proper explainability technique such as:

SHAP

or feature importance.

Then your system can say:

```
Temperature
SHAP contribution: +0.31

Vibration
```

```
SHAP contribution: +0.24
```

```
Load
```

```
SHAP contribution: +0.19
```

You can convert those contributions into an understandable explanation.

Then your advisor can see:

| The system isn't just saying "87%"; it can explain why.

That's much stronger.

19. Agent 4 — Security Agent

This is where you introduce the cybersecurity side.

The Security Agent monitors things such as:

```
Unexpected sensor values  
Abnormal communication  
Repeated failed authentication  
Unusual traffic  
Data poisoning indicators  
Model-update anomalies
```

For example:

```
Substation B
```

```
Received model update
```

```
Deviation from normal:  
87%
```

```
Possible malicious/poisoned update
```

Then:

```
Security Agent
```

```
↓
```

```
Flags Substation B
↓
Aggregator excludes/suspends update
↓
Administrator alerted
```

This introduces **Federated Learning Security**.

20. Agent 5 — Decision Agent

This is the "brain" that combines everything.

Suppose:

```
Forecast Agent:
Demand increasing

Fault Agent:
Voltage abnormal

Maintenance Agent:
Transformer T-17 risk = 87%

Security Agent:
No cyber anomaly
```

The Decision Agent produces:

```
DECISION

Priority: HIGH

Recommended Action:
Schedule inspection of Transformer T-17.

Reason:
• High temperature
• Increasing load
• Abnormal vibration
• Historical stress pattern

Confidence:
89%
```

21. Does the AI automatically control the grid?

No.

For an FYP, I strongly recommend:

```
AI
↓
Recommendation
↓
Human Operator
↓
Approve / Reject
```

rather than:

```
AI
↓
Automatically switch transformer
```

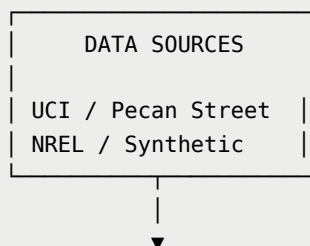
because actual electrical control is a much more serious safety problem.

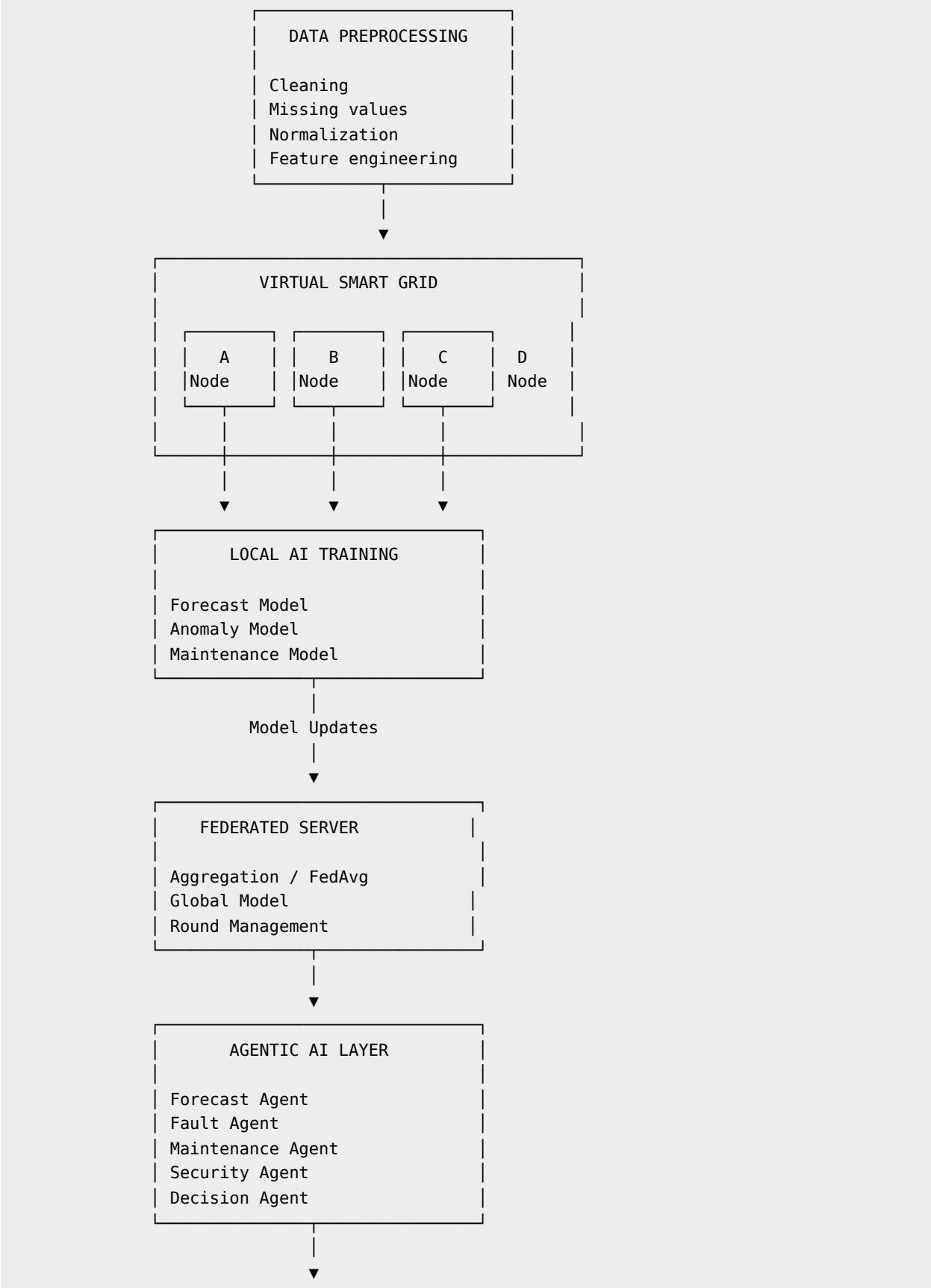
Your project becomes a **Decision Support System**.

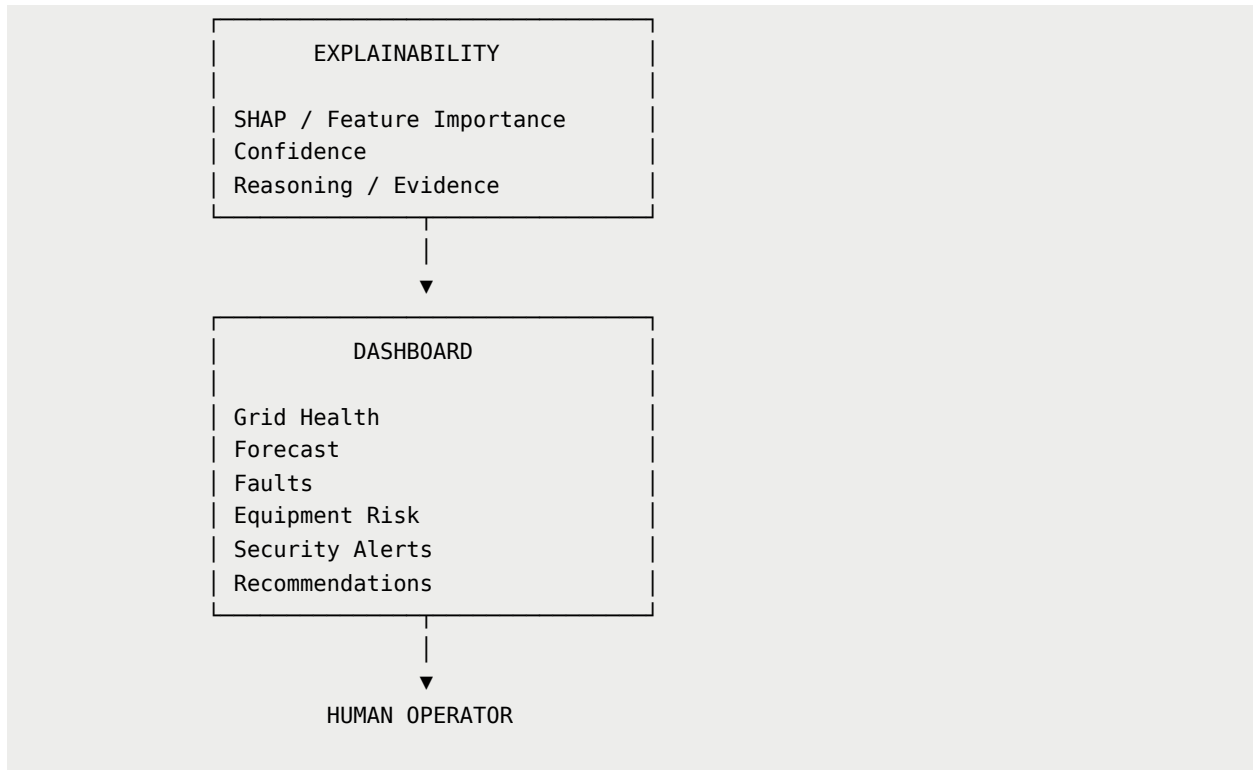
That's safer and much easier to defend.

22. Complete end-to-end architecture

Now let's combine everything.







23. What exactly happens to the data?

Let's follow **one data point**.

Suppose Substation A records:

10:35 AM

Voltage = 231V

Current = 420A

Temperature = 65°C

Vibration = 2.1

Load = 87%

Step 1 — Sensor/data source

Data enters your system.

Sensor



Substation A

Step 2 — preprocessing

Your pipeline checks:

Missing?
Outlier?
Correct units?
Timestamp valid?

Then normalizes the values.

Step 3 — local database

Data goes into:

Substation A database

It is **not immediately uploaded as raw data** to the central FL server.

Step 4 — local ML model

Substation A trains:

Forecast Model
Fault Model
Maintenance Model

using its local dataset.

Step 5 — model update

Instead of:

SEND:

10 million raw sensor records

the node sends something like:

Model Update

to the FL server.

Step 6 — aggregation

The FL server receives:

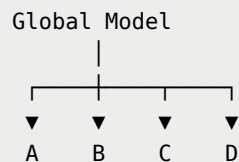
A → Model Update
B → Model Update
C → Model Update
D → Model Update

and produces:

Global Model

Step 7 — global model

The improved model is distributed back:



24. Where does Agentic AI fit?

This is an important distinction.

Federated Learning is not Agentic AI.

They are two different technologies.

Federated Learning

Answers:

| "How can multiple locations collaboratively train AI without sharing raw data?"

Agentic AI

Answers:

| "What should the system do with the information it has?"

So:

```
Federated Learning
  ↓
Better distributed intelligence
  ↓
AI Agents
  ↓
Analyze situation
  ↓
Plan
  ↓
Recommend action
```

25. What makes an agent different from a normal ML model?

A normal model:

```
Input
  ↓
Prediction
```

Example:

```
temperature + vibration + load
      ↓
failure probability = 87%
```

An agent can do:

```
Observe
  ↓
Analyze
  ↓
Use tools
  ↓
Check other information
  ↓
Reason
  ↓
Plan
  ↓
Recommend action
```

For example:

```
Maintenance Agent
|
|— Check transformer telemetry
|
|— Check historical maintenance
|
|— Check current load
|
|— Ask Fault Agent for anomaly status
|
|— Generate recommendation
```

26. Example of actual Agent workflow

Suppose the dashboard receives:

```
Transformer T-17
```

```
Temperature ↑  
Vibration ↑  
Load ↑
```

The orchestrator calls:

```
Maintenance Agent
```

It checks:

```
getTransformerTelemetry(T-17)
```

Then:

```
getMaintenanceHistory(T-17)
```

Then:

```
getCurrentLoad(T-17)
```

Then:

```
getFaultAnalysis(T-17)
```

Then it produces:

```
Risk = HIGH  
  
Recommended action:  
Inspect Transformer T-17  
within the next maintenance cycle.
```

That is much closer to genuine **Agentic AI** than simply putting an LLM chatbot on top of the project.

27. Do we actually need an LLM?

Not necessarily for the core prediction.

This is important.

Don't use an LLM for:

Temperature → failure probability

Use an ML model.

For example:

XGBoost
Random Forest
LSTM
GRU
Temporal models
Autoencoder

The LLM can be used for the **agentic reasoning/interface layer**.

For example:

```
ML models
  ↓
Structured results
  ↓
Agent
  ↓
LLM reasoning
  ↓
Human-readable explanation
```

That is a much better architecture.

28. Suggested ML models

Don't try to use 15 algorithms.

Start with a manageable set.

Forecasting

Potential choices:

```
XGBoost
LSTM
GRU
```

For an FYP, you can compare:

```
Baseline: Random Forest/XGBoost
          vs
Deep model: LSTM
```

Anomaly detection

Use:

```
Isolation Forest
```

or:

```
Autoencoder
```

For your first implementation, Isolation Forest is considerably easier.

Predictive maintenance

Use:

```
Random Forest
XGBoost
```

and output:

Failure probability

Explainability

Use:

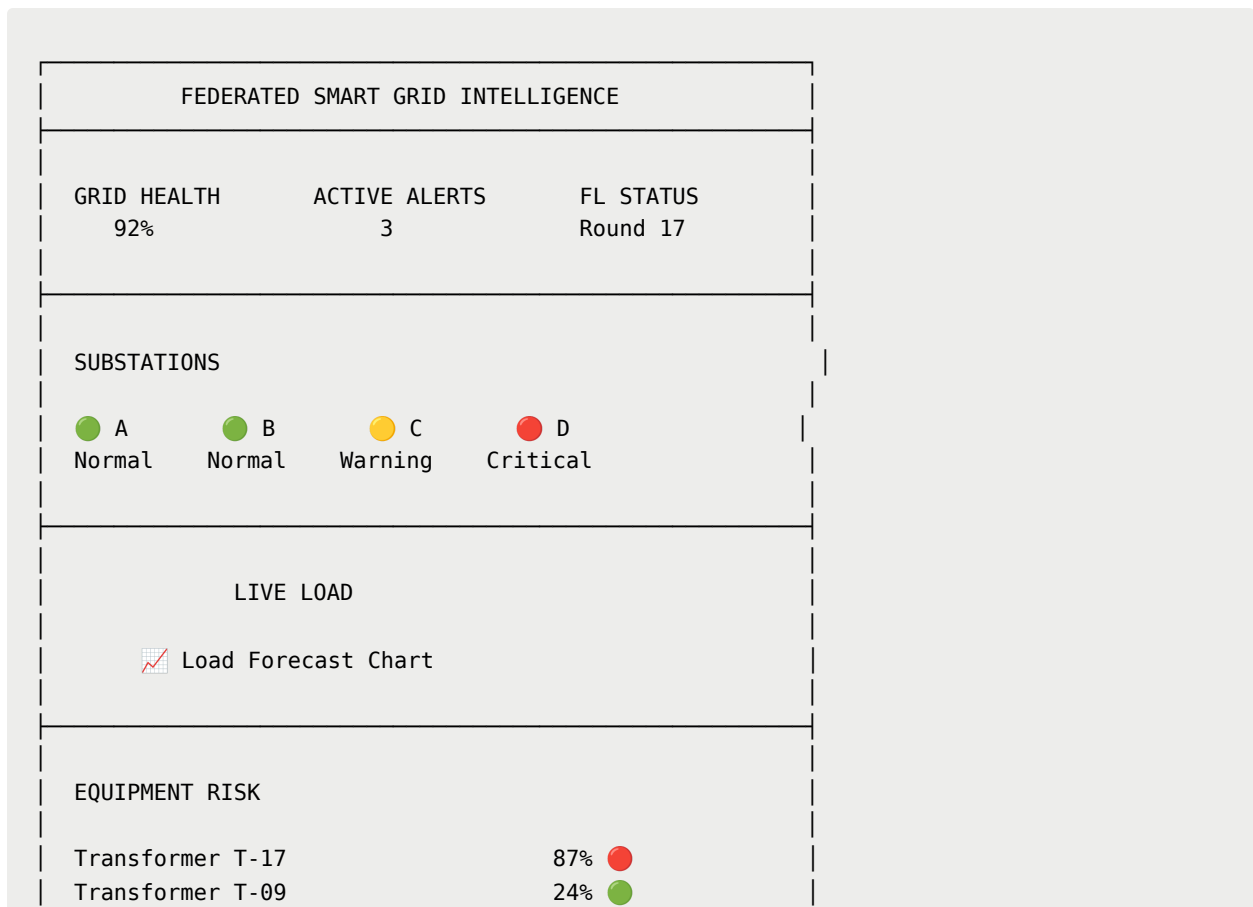
SHAP

29. The dashboard is extremely important

Your advisor should **not have to look at Python terminal output**.

You should create a professional web dashboard.

For example:



Transformer T-21

61% 

30. Substation screen

When the advisor clicks:

Substation C

show:

SUBSTATION C

Voltage	231.4 V
Current	412 A
Power	94.2 kW
Frequency	49.98 Hz

Temperature	82°C
Vibration	4.2 mm/s
Transformer Load	91%

Status:

⚠ WARNING

Then:

AI ANALYSIS

Anomaly Score:
0.83

Primary concern:
Transformer overheating

31. Predictive maintenance screen

Click:

Transformer T-17

Display:

TRANSFORMER HEALTH

Health Score



Failure Probability

87%

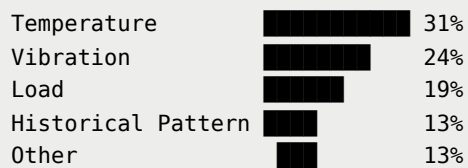
Risk Level

HIGH

Then:

Explainability

Why?



Then:

RECOMMENDATION

Schedule inspection.

Priority:

HIGH

32. Federated Learning dashboard

This is something that can really impress your advisors.

Create a page:

FEDERATED LEARNING

Round: 17

Participating Nodes:

A ✓
B ✓
C ✓
D ✓

Global Model Accuracy

Round 1	71%
Round 5	78%
Round 10	84%
Round 15	88%
Round 17	91%

And:

Local Training

A → completed
B → completed
C → completed
D → completed

Aggregation → completed

Global Model → updated

33. Demonstrate privacy visually

This could be one of your strongest demonstrations.

Show:

SUBSTATION A

Local Records:
2,000,000

Raw data sent to central server:

0 records

Then:

Model Update:
✓ Sent

Your advisor immediately understands the purpose of Federated Learning.

34. Security demonstration

You can intentionally inject a malicious/abnormal model update.

Normal:

A → normal update
B → normal update
C → normal update
D → normal update

Then simulate:

C → malicious update

Security Agent:

⚠ Suspicious Model Update

Node:
C

```
Deviation:
91%

Action:
Update rejected
```

This gives you an actual **cybersecurity experiment**, instead of merely claiming your project has cybersecurity.

35. This is where Trustworthy AI comes in

Since you're interested in **Trustworthy AI**, this project can incorporate it naturally.

You can define:

1. Explainability

```
Why did AI predict failure?
```

→ SHAP

2. Confidence

```
Failure probability = 87%
Confidence = high
```

3. Human oversight

```
AI recommendation
  ↓
Human approval
```

4. Privacy

```
Raw data stays local
```

5. Security

```
Malicious model update detection
```

6. Auditability

Store:

```
Prediction  
Timestamp  
Model version  
Agent used  
Recommendation  
Human decision
```

So you can claim:

The platform incorporates privacy, explainability, human oversight, security and auditability principles for trustworthy AI.

That's a much stronger academic story.

36. Your database design

You don't need one giant database.

You could use:

```
PostgreSQL
```

for central application data.

Tables could include:

```
users  
substations  
transformers  
sensor_readings  
maintenance_records
```

```
predictions
alerts
recommendations
federated_rounds
model_updates
audit_logs
```

Example:

substations

```
id
name
location
status
created_at
```

transformers

```
id
substation_id
transformer_name
capacity
installation_date
status
```

sensor_readings

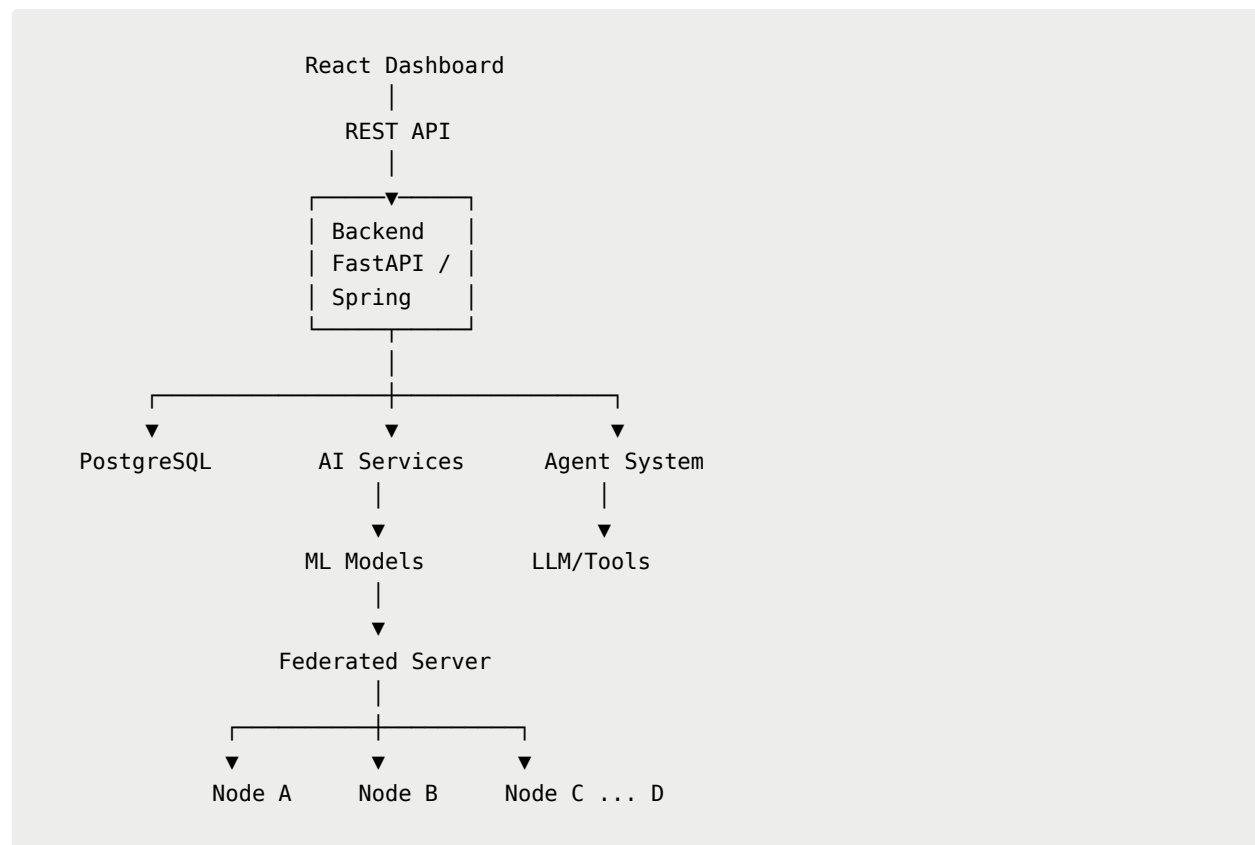
```
id
transformer_id
timestamp
voltage
current
temperature
vibration
load
frequency
```

predictions

```
id
transformer_id
model_version
failure_probability
risk_level
timestamp
```

37. Backend architecture

Since you're a Software Engineering student, a clean architecture would be:



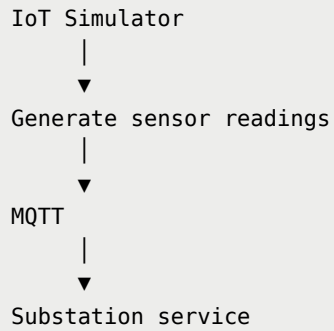
You could implement the AI/FL components in Python and expose them through APIs while the main application handles authentication, dashboard, users, alerts and audit logs.

38. How do we simulate IoT?

You don't need physical sensors.

Create an **IoT simulator**.

For example:



It can generate:

- Voltage
- Current
- Temperature
- Vibration
- Load
- Frequency

every few seconds.

Example:

```
{
  "substation": "SUB-C",
  "transformer": "T-17",
  "temperature": 82.4,
  "vibration": 4.1,
  "voltage": 231.2,
  "current": 412,
  "load": 91.2
}
```

This makes the dashboard appear **live**.

39. Your FYP doesn't need physical IoT hardware

This is important.

You can present:

“We implement a virtual IoT layer to emulate smart-grid sensor telemetry.”

So:

```
Real sensor
  ↓
MQTT
  ↓
Backend
```

becomes:

```
Sensor Simulator
  ↓
MQTT
  ↓
Backend
```

From the software architecture perspective, the downstream system can remain essentially the same.

40. How do you create faults for the demo?

This is actually very useful.

Create a **Fault Injection Engine**.

Normal:

```
Temperature = 60°C
```

Fault scenario:

Temperature gradually increases:

60
63
67
72
78
85
91

At the same time:

Vibration:

1.2
1.4
1.8
2.5
3.4
4.8
6.1

Your AI sees the pattern.

Eventually:

Risk → HIGH

and your dashboard changes:

 NORMAL

↓

 WARNING

↓

 CRITICAL

This makes your final demonstration much more impressive than simply showing a static CSV.

41. Complete demo scenario for your advisors

I would actually design your final presentation around **one story**.

Start with:

```
All substations normal.
```

Dashboard:

A ●
B ●
C ●
D ●

Then start the IoT simulator.

Step 1

Demand begins increasing at Substation C.

```
Load:  
72%  
76%  
81%  
86%  
91%
```

Forecast Agent detects:

```
Expected high demand.
```

Step 2

Transformer temperature begins increasing.

61°C
66°C
71°C
78°C
84°C

Fault Agent:

Anomaly detected.

Step 3

Vibration begins increasing.

1.2
1.8
2.7
3.9
5.1

Maintenance Agent:

Failure probability:
87%

Step 4

Explainability appears.

Temperature	+31%
Vibration	+24%

Load	+19%
Historical	+13%

Step 5

Decision Agent produces:

HIGH PRIORITY

Transformer T-17 requires inspection.

Recommended action:

Schedule preventive maintenance.

Step 6

Now demonstrate cybersecurity.

Inject a suspicious federated update from another node.

Security Agent:

Suspicious update detected.

Node: B

Deviation: 91%

Action:

REJECT UPDATE

Step 7

Show Federated Learning.

Round 17

A ✓

B ⚠ rejected

C ✓

D ✓

Global model updated.

This gives you one demonstration covering:

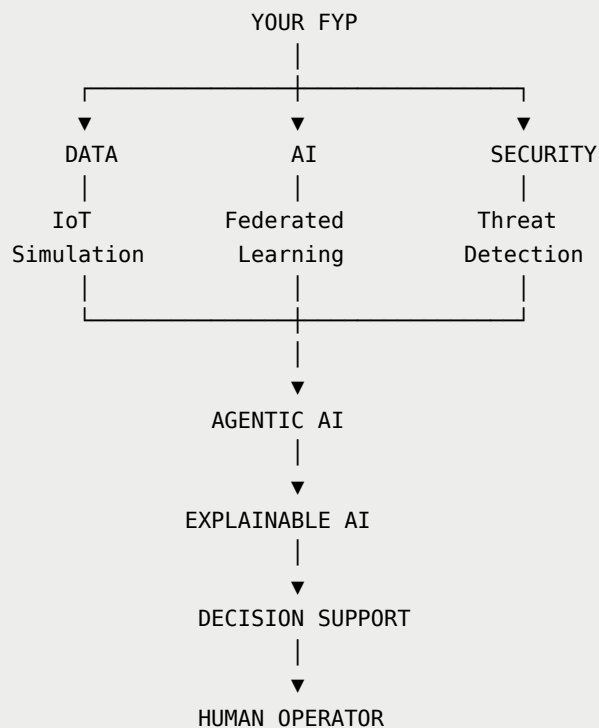
IoT + ML + Federated Learning + Agentic AI + Explainable AI + Cybersecurity + Human-in-the-loop.

42. What your advisor sees

Instead of saying:

| "We made an AI model."

You're demonstrating:



That's a much more substantial FYP.

43. Suggested project modules

I would divide the development into these modules:

Module 1 — Data Management

- Dataset ingestion
- Data cleaning
- Feature engineering
- Data partitioning

Module 2 — Smart Grid Simulator

- Substations
- Sensors
- Transformers
- Real-time telemetry
- Fault injection

Module 3 — ML

- Load forecasting
- Anomaly detection
- Predictive maintenance

Module 4 — Federated Learning

- FL clients
- Local training
- Aggregation
- Global model
- Federated rounds

Module 5 — Agentic AI

- Forecast Agent
- Fault Agent

Maintenance Agent
Security Agent
Decision Agent

Module 6 — Trustworthy AI

Explainability
Confidence
Audit logs
Human approval
Privacy

Module 7 — Dashboard

Grid overview
Substation monitoring
Forecasting
Equipment health
Security
FL rounds
Recommendations

44. Suggested technology stack

A practical stack could be:

Layer	Technology
Frontend	React + Vite
Dashboard	React + Recharts
Main backend	FastAPI or Spring Boot
ML	Python + Scikit-learn
Deep Learning	PyTorch
Federated Learning	Flower
Database	PostgreSQL
Real-time communication	WebSocket

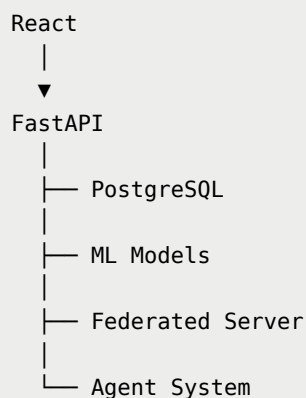
Layer	Technology
IoT messaging	MQTT
Agent orchestration	Python
LLM	OpenAI API / another suitable LLM
Explainability	SHAP
Authentication	JWT + RBAC
Deployment	Docker
Monitoring	Prometheus/Grafana optional

You **don't need every technology** on day one.

45. A simpler implementation architecture for your team

If you're new to this domain, don't build everything simultaneously.

Start with:



Then add:

MQTT

Then:

Security

Then:

Explainability

46. Your minimum viable FYP

Don't let the project become too large.

The **minimum complete version** should be:

```
4 simulated substations
  ↓
local datasets
  ↓
local ML training
  ↓
Federated Learning
  ↓
global model
  ↓
3 AI agents
  ↓
explainability
  ↓
dashboard
  ↓
human recommendation
```

The three essential agents can initially be:

```
Forecast Agent
Fault Agent
Maintenance Agent
```

Then add:

Security Agent
Decision Agent

once the core works.

47. What you should NOT try to build

I strongly recommend **not** attempting:

- ✗ Real electrical-grid control
- ✗ Physical high-voltage hardware
- ✗ A huge LLM trained from scratch
- ✗ 20 different AI models
- ✗ Fully autonomous grid operation
- ✗ Real-time control of electrical equipment
- ✗ Massive distributed cloud infrastructure
- ✗ A complicated digital twin of an entire country

Your FYP should demonstrate the **concept convincingly**, not attempt to replace an actual utility control system.

48. What makes the research component strong?

Your FYP can contain actual experiments.

For example:

Experiment 1 — Centralized vs Federated Learning

Compare:

Centralized ML
vs

Federated ML

Measure:

Accuracy
Precision
Recall
F1
Training time
Communication overhead

Experiment 2 — Different numbers of substations

2 nodes
4 nodes
8 nodes

Measure:

Accuracy
Convergence
Training time

Experiment 3 — Non-IID data

Compare:

IID
vs
Non-IID

Experiment 4 — Malicious participant

Inject:

Normal clients
+
malicious client

Measure:

Global model degradation
Detection rate
False positives

Experiment 5 — Explainability

Evaluate whether the system can identify the major contributors to the predicted failure.

49. Your final research question could be something like

Can federated learning combined with agentic AI provide accurate, privacy-preserving and explainable smart-grid monitoring and predictive maintenance across distributed substations?

Then you have measurable objectives.

Objective 1

Develop distributed smart-grid data nodes.

Objective 2

Develop ML models for:

Forecasting
Anomaly detection
Predictive maintenance

Objective 3

Implement federated learning.

Objective 4

Develop intelligent agents for analysis and decision support.

Objective 5

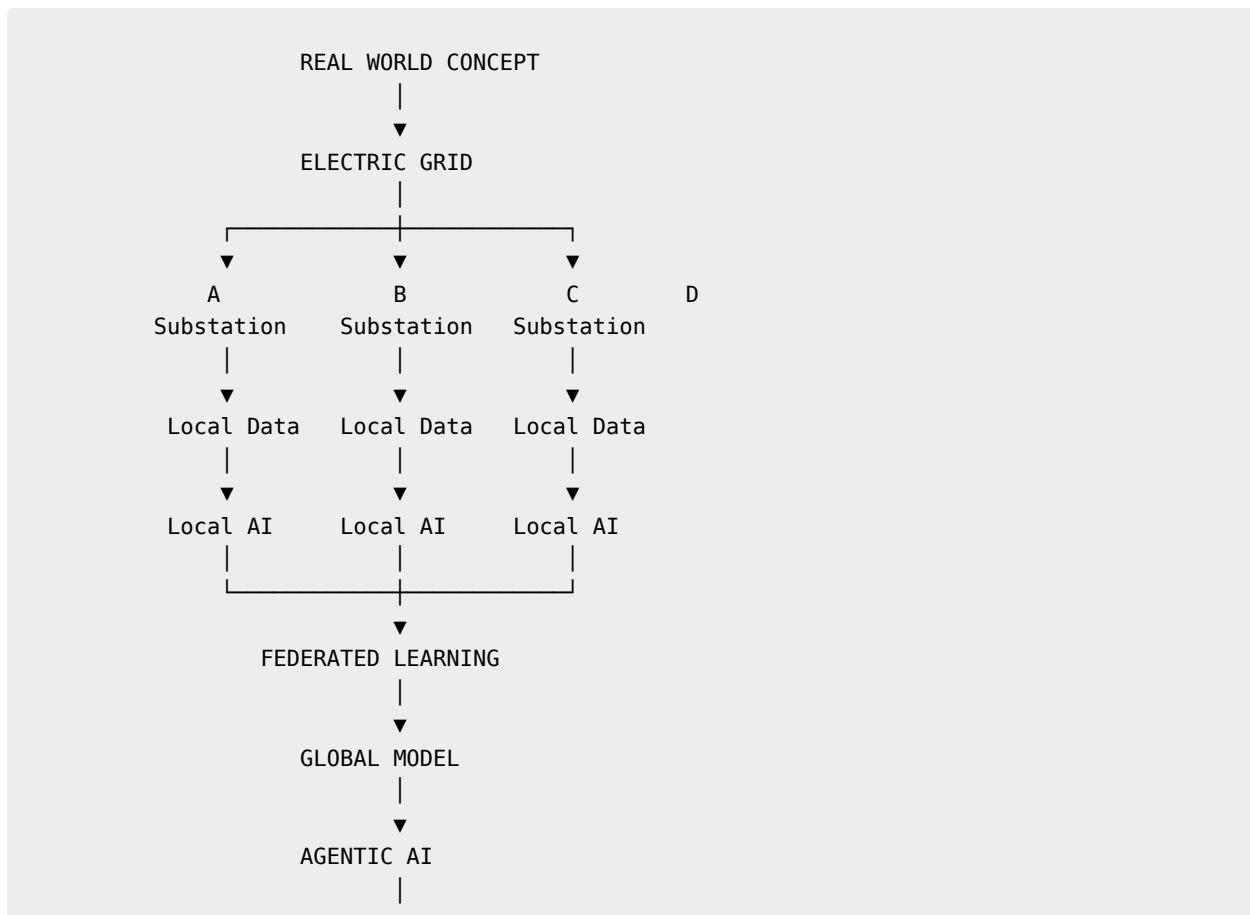
Implement explainability and human oversight.

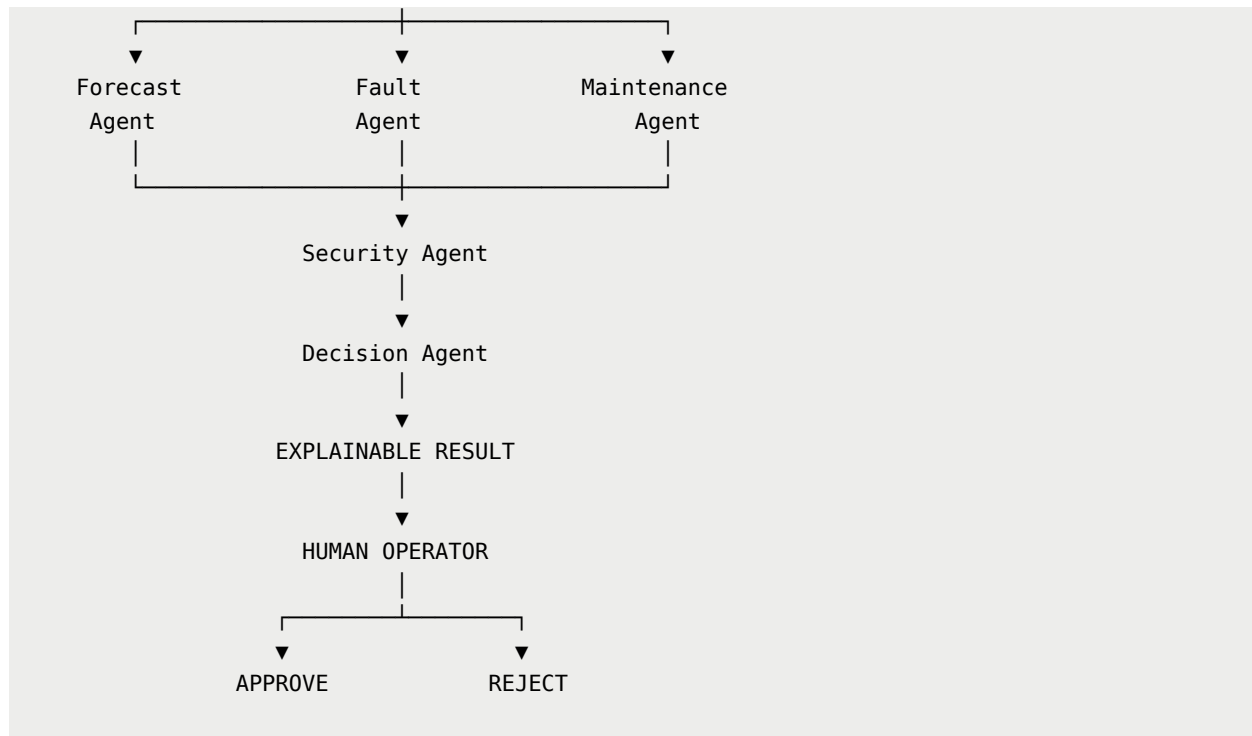
Objective 6

Evaluate resilience against suspicious/malicious model updates.

50. The complete mental model

If you remember only one diagram, remember this:





51. And the most important thing: what are you actually claiming?

You are **not** claiming:

"We built a real smart grid."

You are claiming:

"We developed a simulated distributed smart-grid intelligence platform that uses federated learning to collaboratively train models across independent substations while keeping raw data local, and uses specialized AI agents to perform forecasting, anomaly detection, predictive maintenance, security analysis, explainability and human-supervised decision support."

That is a **very defensible FYP statement.**

And it is substantially more interesting than:

"We made a smart-grid chatbot."

52. Recommended final FYP architecture

If I were structuring this specifically for your FYP, I would settle on this:

